UNIVERSITÉ DE NAMUR

Econométrie

Introduction à R

Mathilde Col & Jérémy Do Nascimento Miguel
Université de Namur
2026-09-16

Ce cours

- Introduction à R
- Bases de la programmation
- Statistiques descriptives et régressions
- Gestion des données dans R
- Communiquer ses résultats dans R (visualisations, tableaux)



Cette séance

- Qu'est-ce que R ?
- Démarrer R et RStudio
- R comme calculatrice : opérations de base
- Organiser un projet
- Types de données
- Opérateurs logiques
- Objets
- Structures de données : vecteurs, matrices, data frames
- Installer des packages
- Sous-ensembles (*subsetting*) de data frames



R & RStudio

R

Qu'est-ce que R ?

- Un langage de programmation et un environnement logiciel pour le calcul statistique
- Il est **gratuit** et **open-source**
- Vous pouvez le télécharger sur : http://www.r-project.org/

    1. téléchargez R
    2. choisissez le miroir de téléchargement (peu importe lequel)
    3. choisissez votre système d'exploitation
    4. sélectionnez la version dont vous avez besoin



R

Qu'est-ce que R ?

- Un langage de programmation et un environnement logiciel pour le calcul statistique
- Il est **gratuit** et **open-source**
- Vous pouvez le télécharger sur : http://www.r-project.org/

    1. téléchargez R
    2. choisissez le miroir de téléchargement (peu importe lequel)
    3. choisissez votre système d'exploitation
    4. sélectionnez la version dont vous avez besoin

- Ou directement les liens pour R 4.5.1 (vous pouvez télécharger une version plus récente, mais nous utiliserons celle-ci) :

    - Windows
    - Apple Silicon (M1, M2, ...) / Intel Mac
    - Linux Debian / Fedora / Redhat / Ubuntu



R

Qu'est-ce que R ?

- Un langage de programmation et un environnement logiciel pour le calcul statistique
- Il est **gratuit** et **open-source**
- Vous pouvez le télécharger sur : http://www.r-project.org/

  1. téléchargez R
  2. choisissez le miroir de téléchargement (peu importe lequel)
  3. choisissez votre système d'exploitation
  4. sélectionnez la version dont vous avez besoin

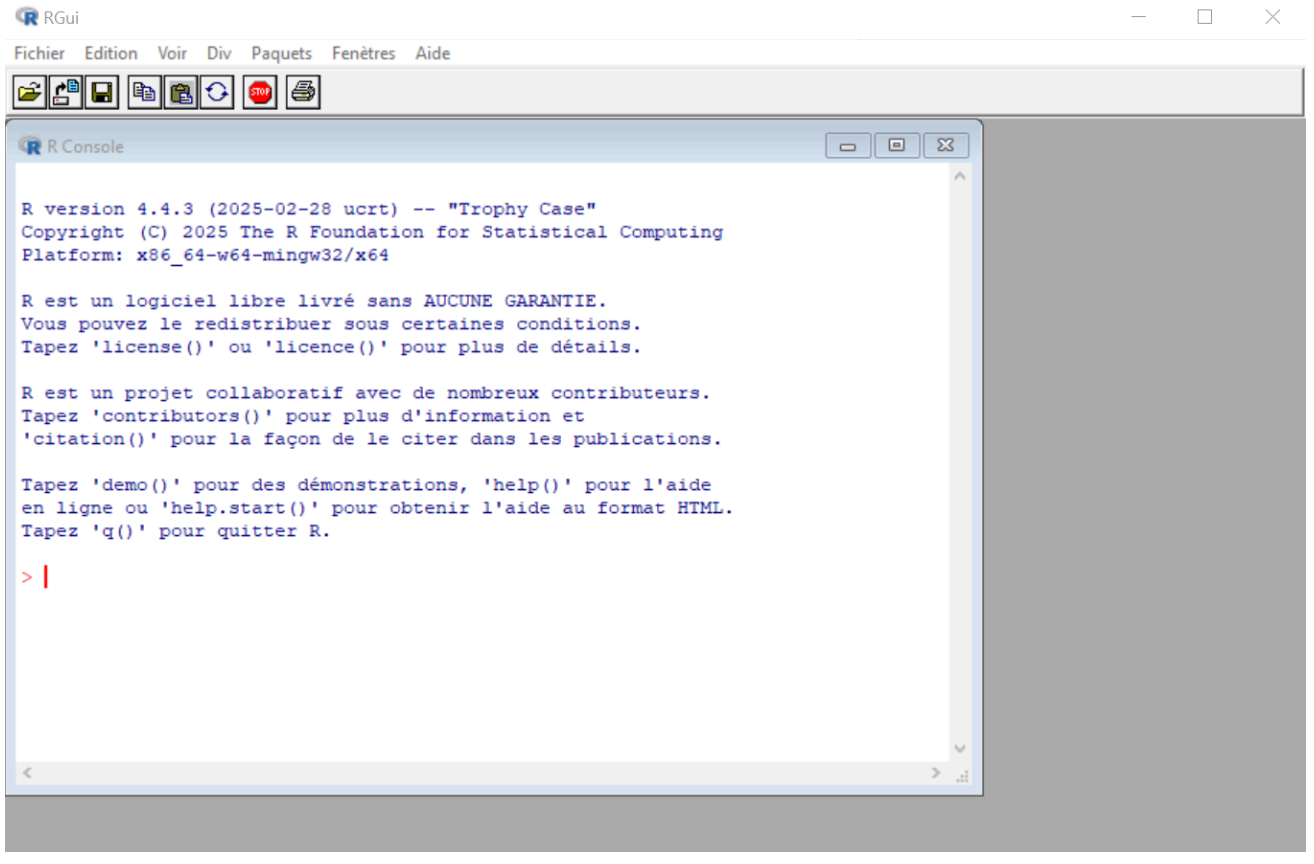
- Ou directement les liens pour R 4.5.1 (vous pouvez télécharger une version plus récente, mais nous utiliserons celle-ci) :

  - Windows
  - Apple Silicon (M1, M2, ...) / Intel Mac
  - Linux Debian / Fedora / Redhat / Ubuntu

- Pour Windows, installez également Rtools : Rtools 4.5



R - Interface graphique



RStudio

- RStudio, un IDE (environnement de développement intégré)
  - on peut écrire du code R sans RStudio
  - on pourrait même écrire du R dans Notepad, Word ou PowerPoint
  - mais vous devriez écrire votre code R dans RStudio



RStudio

- RStudio, un IDE (environnement de développement intégré)
  - on peut écrire du code R sans RStudio
  - on pourrait même écrire du R dans Notepad, Word ou PowerPoint
  - mais vous devriez écrire votre code R dans RStudio
- L'entreprise qui maintient RStudio s'est rebaptisée Posit

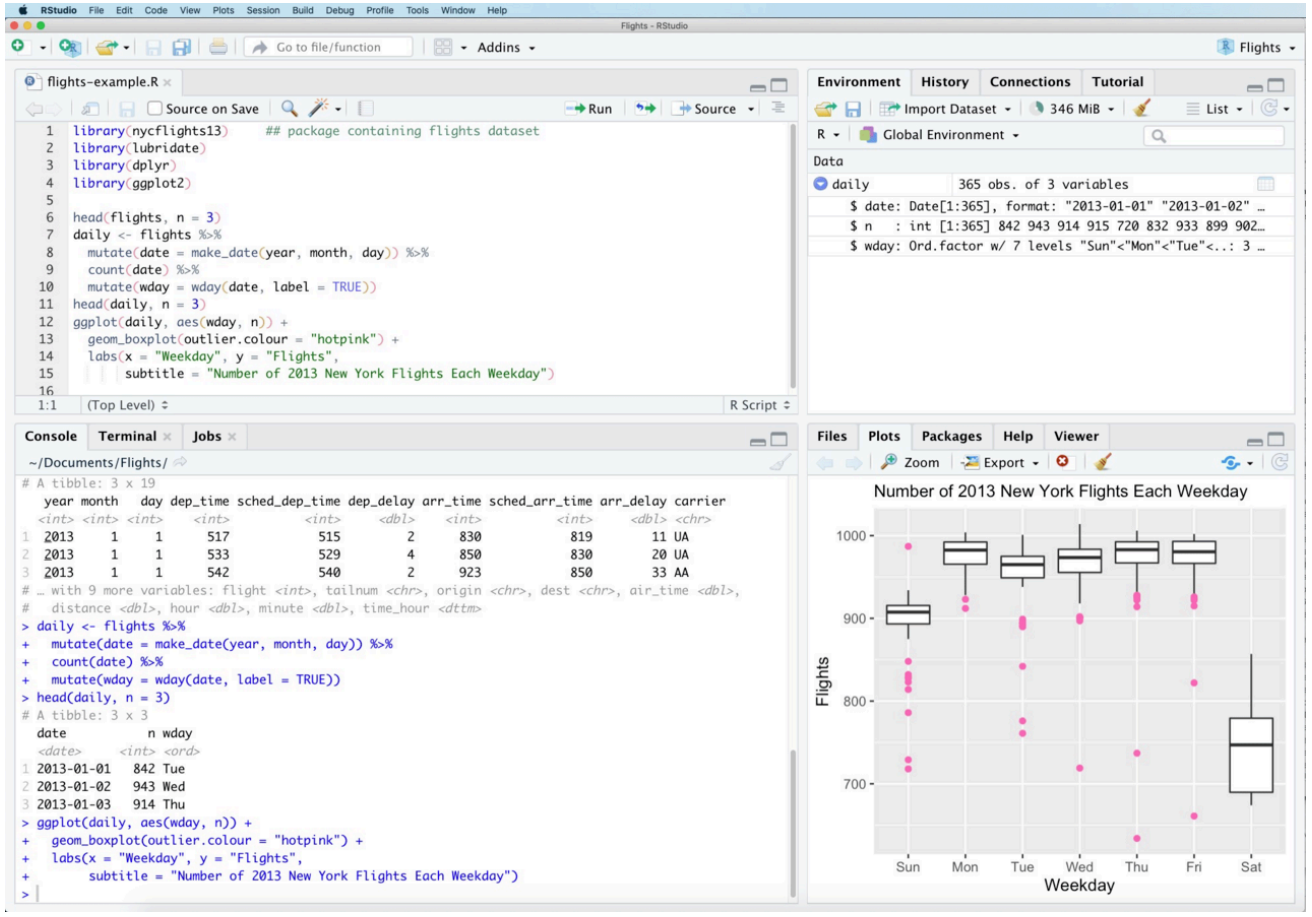


RStudio

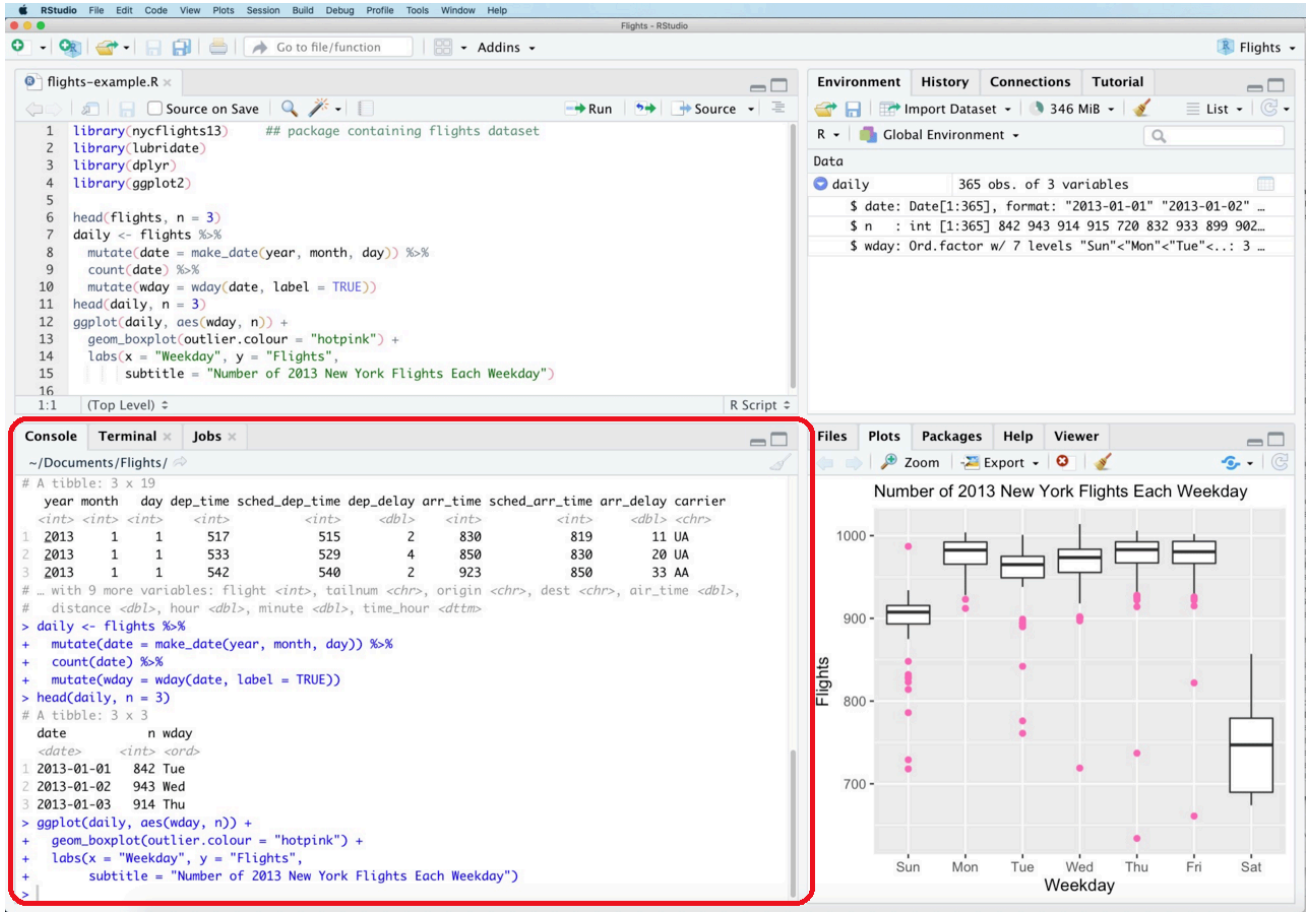
- RStudio, un IDE (environnement de développement intégré)
  - on peut écrire du code R sans RStudio
  - on pourrait même écrire du R dans Notepad, Word ou PowerPoint
  - mais vous devriez écrire votre code R dans RStudio
- L'entreprise qui maintient RStudio s'est rebaptisée Posit
- Vous pouvez télécharger RStudio ici : https://posit.co/downloads/



RStudio


RStudio File Edit Code View Plots Session Build Debug Profile Tools Window Help
Flights - RStudio
Go to file/function
Addins
Flights
flights-example.R
Source on Save
Run
Source
1 library(nycflights13) ## package containing flights dataset
2 library(lubridate)
3 library(dplyr)
4 library(ggplot2)
5
6 head(flights, n = 3)
7 daily <- flights %>%
8 mutate(date = make_date(year, month, day)) %>%
9 count(date) %>%
10 mutate(wday = wday(date, label = TRUE))
11 head(daily, n = 3)
12 ggplot(daily, aes(wday, n)) +
13 geom_boxplot(outlier.colour = "hotpink") +
14 labs(x = "Weekday", y = "Flights",
15 subtitle = "Number of 2013 New York Flights Each Weekday")
16
1:1 (Top Level)
R Script
Environment History Connections Tutorial
Import Dataset
346 MiB
List
R
Global Environment
Data
daily 365 obs. of 3 variables
$ date: Date[1:365], format: "2013-01-01" "2013-01-02" …
$ n : int [1:365] 842 943 914 915 720 832 933 899 902…
$ wday: Ord.factor w/ 7 levels "Sun"<"Mon"<"Tue"<..: 3 …
Console Terminal Jobs
~/Documents/Flights/
A tibble: 3 x 19
year month day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay carrier
<int> <int> <int> <int> <int> <dbl> <int> <int> <dbl> <chr>
1 2013 1 1 517 515 2 830 819 11 UA
2 2013 1 1 533 529 4 850 830 20 UA
3 2013 1 1 542 540 2 923 850 33 AA
… with 9 more variables: flight <int>, tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>,
distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dttm>
> daily <- flights %>%
+ mutate(date = make_date(year, month, day)) %>%
+ count(date) %>%
+ mutate(wday = wday(date, label = TRUE))
> head(daily, n = 3)
A tibble: 3 x 3
date n wday
<date> <int> <ord>
1 2013-01-01 842 Tue
2 2013-01-02 943 Wed
3 2013-01-03 914 Thu
> ggplot(daily, aes(wday, n)) +
+ geom_boxplot(outlier.colour = "hotpink") +
+ labs(x = "Weekday", y = "Flights",
+ subtitle = "Number of 2013 New York Flights Each Weekday")
>
Files Plots Packages Help Viewer
Zoom
Export
Number of 2013 New York Flights Each Weekday
1000
900
800
700
Flights
Sun
Mon
Tue
Wed
Thu
Fri
Sat
Weekday

RStudio - Console



La console R

Vous pouvez rapidement tester R comme une calculatrice

```
# Addition
5 + 5

# Soustraction
5 - 5

# Multiplication
3 * 5

# Division
10 / 2

# Exposant
5^3
```



Les scripts R

Pour sauvegarder toutes vos commandes, vous devez utiliser des scripts R

- des fichiers texte avec l'extension `.R`
- similaires aux do-files Stata (.do) ou aux scripts Python (.py)
- nouveau fichier via le bouton en haut à gauche



Les scripts R

Pour sauvegarder toutes vos commandes, vous devez utiliser des scripts R

- des fichiers texte avec l'extension `.R`
- similaires aux do-files Stata (.do) ou aux scripts Python (.py)
- nouveau fichier via le bouton en haut à gauche

Pour exécuter votre code

- Exécutez la ligne courante ou la sélection avec le bouton *Run*
- Ou utilisez ctrl/cmd + enter



Les scripts R

Pour sauvegarder toutes vos commandes, vous devez utiliser des scripts R

- des fichiers texte avec l'extension `.R`
- similaires aux do-files Stata (.do) ou aux scripts Python (.py)
- nouveau fichier via le bouton en haut à gauche

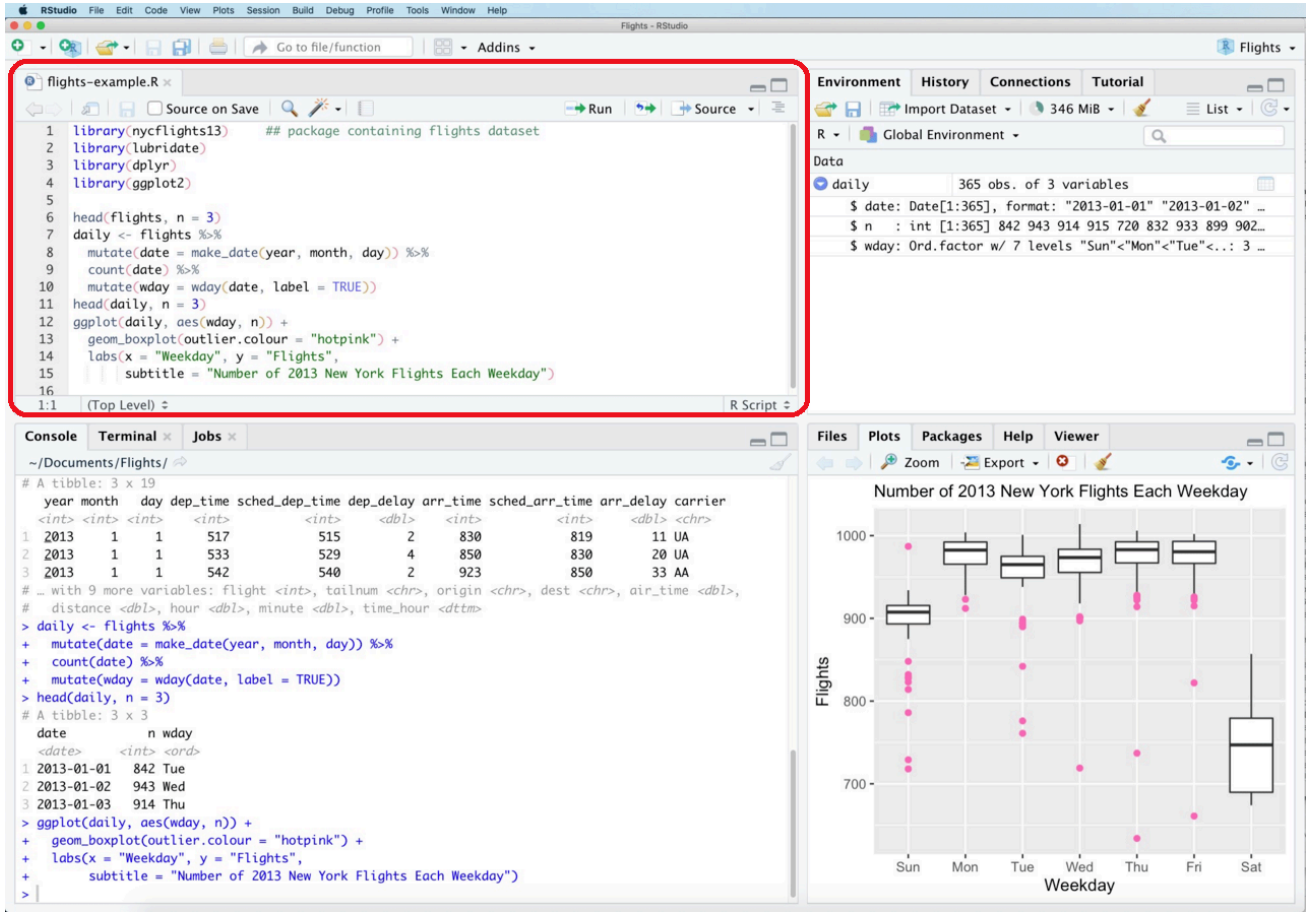
Pour exécuter votre code

- Exécutez la ligne courante ou la sélection avec le bouton *Run*
- Ou utilisez ctrl/cmd + enter

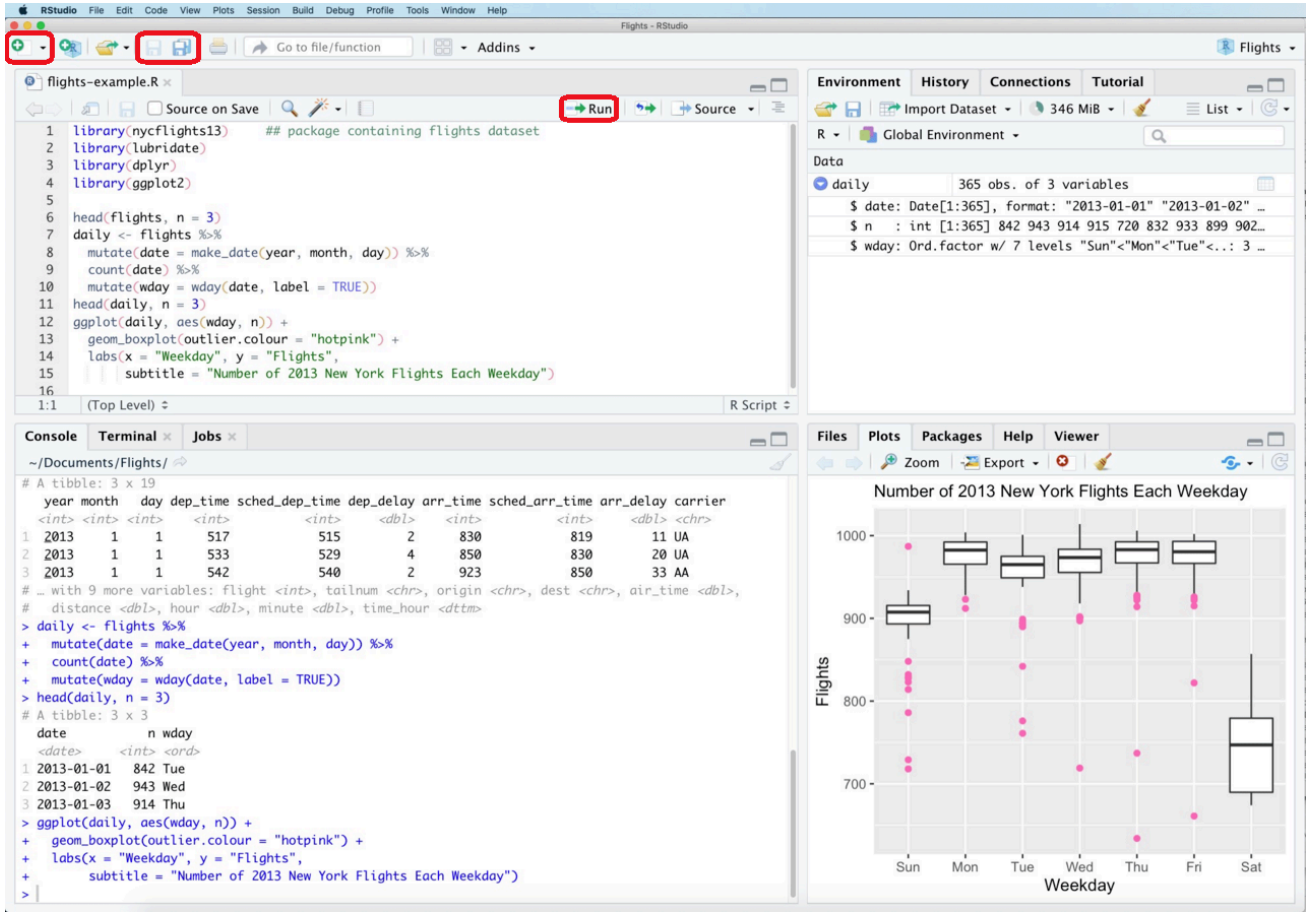
Vous pouvez ajouter des commentaires avec #, comme // et * dans Stata



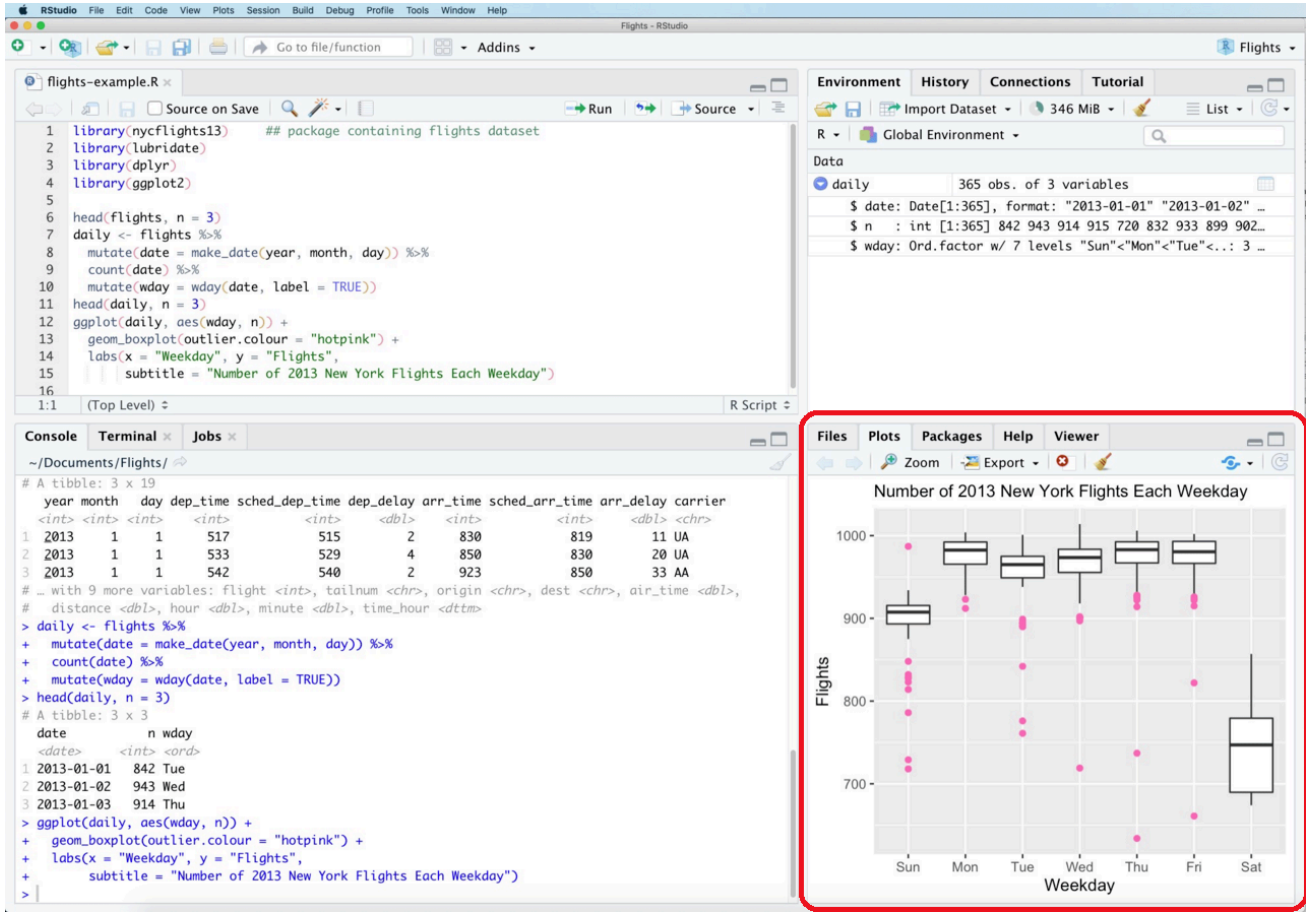
RStudio - Scripts



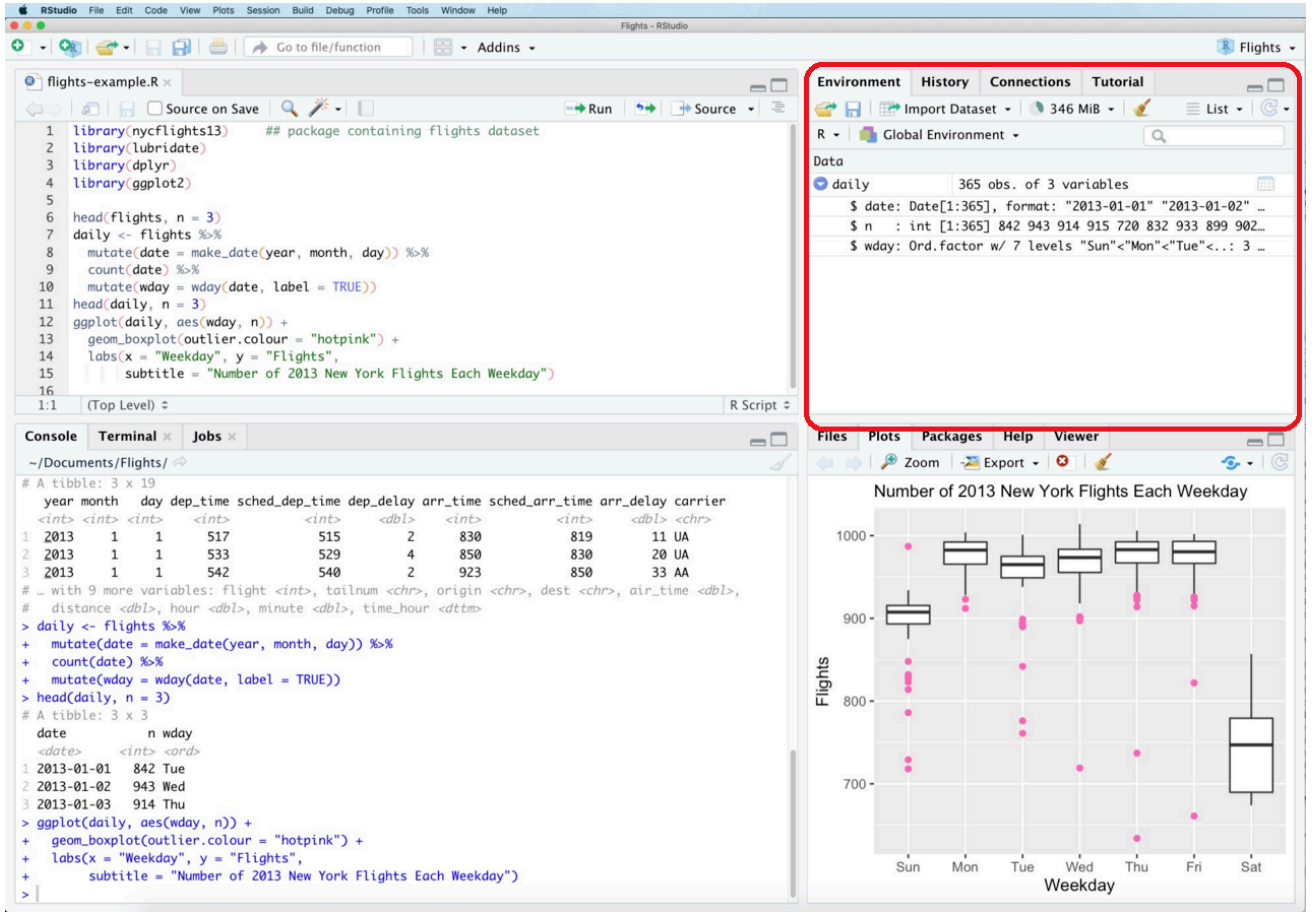
RStudio - Exécution



RStudio - Graphiques, aide et plus



RStudio - Environnement, historique et plus



Organisation d'un projet

Organisation d'un projet

Pour de plus gros projets, comme votre mémoire/thèse :

- votre répertoire de projet
  - data/
    - 1_raw/
    - 2_clean/
  - code/
    - 1_clean.R
    - 2_analyze.R
    - requirements.R
    - main.R
  - docs/
    - sources.md
  - results/
    - figures/
    - tables/
  - README.md



Quelques conseils

- Vos données, votre code et tout le reste doivent être stockés dans le même dossier



Quelques conseils

- Vos données, votre code et tout le reste doivent être stockés dans le même dossier
- Gardez une trace de toutes vos sources de données (+ codebooks) dans un fichier



Quelques conseils

- Vos données, votre code et tout le reste doivent être stockés dans le même dossier
- Gardez une trace de toutes vos sources de données (+ codebooks) dans un fichier
- Ne modifiez **jamais** les données brutes



Quelques conseils

- Vos données, votre code et tout le reste doivent être stockés dans le même dossier
- Gardez une trace de toutes vos sources de données (+ codebooks) dans un fichier
- Ne modifiez **jamais** les données brutes
- Découpez votre code en plusieurs fichiers



Quelques conseils

- Vos données, votre code et tout le reste doivent être stockés dans le même dossier
- Gardez une trace de toutes vos sources de données (+ codebooks) dans un fichier
- Ne modifiez **jamais** les données brutes
- Découpez votre code en plusieurs fichiers
- Exécuter l'intégralité du fichier doit toujours produire le même résultat



Quelques conseils

- Vos données, votre code et tout le reste doivent être stockés dans le même dossier
- Gardez une trace de toutes vos sources de données (+ codebooks) dans un fichier
- Ne modifiez **jamais** les données brutes
- Découpez votre code en plusieurs fichiers
- Exécuter l'intégralité du fichier doit toujours produire le même résultat
- Le fichier README résume le fonctionnement de votre projet
    - Template pour les packages de réplication en sciences sociales



Quelques conseils

- Vous pouvez créer des sections dans votre code avec # et au moins 4 — ou = (comme les sections ** # dans Stata)

```
# Section 1 ----
## Sous-section 1.1 ----
## Sous-section 1.2 ----

# Section 2 ----
## Sous-section 2.1 ----
```



Chemins d'accès (*paths*)

- Répertoire de travail (*working directory*)
  - là où vous travaillez
  - là où R va chercher les fichiers et sauvegarder vos éléments

```
# Obtenir le répertoire de travail
getwd()
setwd("chemin_vers_le_dossier") # Changer le répertoire de travail
```



Chemins d'accès (*paths*)

- Répertoire de travail (*working directory*)
  - là où vous travaillez
  - là où R va chercher les fichiers et sauvegarder vos éléments

```
# Obtenir le répertoire de travail
getwd()
setwd("chemin_vers_le_dossier") # Changer le répertoire de travail
```

- Chemin absolu
  - "C:/Users/nom_utilisateur/documents/session_1/code.R"



Chemins d'accès (*paths*)

- Répertoire de travail (*working directory*)
  - là où vous travaillez
  - là où R va chercher les fichiers et sauvegarder vos éléments

```
# Obtenir le répertoire de travail
getwd()
setwd("chemin_vers_le_dossier") # Changer le répertoire de travail
```

- Chemin absolu
  - "C:/Users/nom_utilisateur/documents/session_1/code.R"
- Chemin relatif, si vous êtes déjà dans le répertoire *document*
  - "session_1/code.R"

```
# Changer le répertoire de travail
setwd("chemin_vers_le_dossier")
```



Chemins d'accès (*paths*)

- Répertoire de travail (*working directory*)
  - là où vous travaillez
  - là où R va chercher les fichiers et sauvegarder vos éléments

```
# Obtenir le répertoire de travail
getwd()
setwd("chemin_vers_le_dossier") # Changer le répertoire de travail
```

- Chemin absolu
  - "C:/Users/nom_utilisateur/documents/session_1/code.R"
- Chemin relatif, si vous êtes déjà dans le répertoire *document*
  - "session_1/code.R"

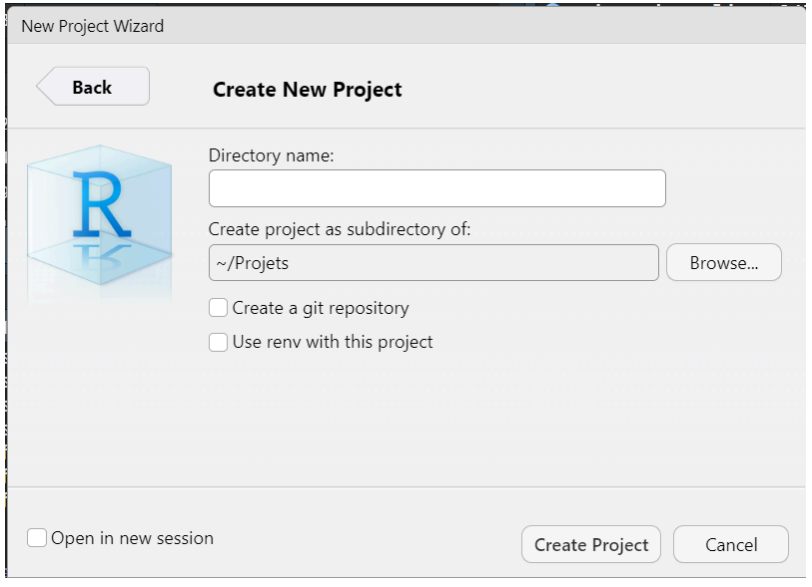
```
# Changer le répertoire de travail
setwd("chemin_vers_le_dossier")
```



- À partir de là, vous pouvez utiliser des chemins relatifs

Projet R (*R project*)

- Les R-projects définissent automatiquement le répertoire de travail
- Fichier → New Project



- Crée un fichier `.Rproj` dans le dossier, avec le nom du dossier



Organisation d'un projet

Pour de plus gros projets, comme votre mémoire de master :

- votre répertoire de projet
  - data/
    - 1_raw/
    - 2_clean/
  - code/
    - 1_clean.R
    - 2_analyze.R
    - requirements.R
    - main.R
  - docs/
    - sources.md
  - results/
    - figures/
    - tables/
  - README.md
  - `nom_du_projet.Rproj`



Exercice 1

05:00

1. Créez un répertoire pour stocker tous vos fichiers relatifs à ce cours
2. Créez un sous-répertoire en tant que R-project pour la séance 1
3. Ouvrez et sauvegardez un nouveau script R dans le répertoire de la séance 1
4. Vérifiez le répertoire de travail
5. Changez-le vers le répertoire de la séance 1 si nécessaire



Les objets en R

Types de données de base

Similaires à d'autres langages de programmation

- Numeric (ou Double)
  - le type par défaut pour les nombres
  - 2, 2.3, 0, -12.8, -Inf, Inf, NaN
- Integer
  - des nombres entiers : 1, 2, 0, -1, ...
  - notation 1L, 2L, ... pour garantir qu'un nombre est un entier et non un double
  - les entiers sont un sous-ensemble des numeric
- Logical
  - deux valeurs : `TRUE` / `FALSE`
- Character (chaînes de caractères)
  - "Éthiopie", "Addis-Abeba"



Types de données de base

Similaires à d'autres langages de programmation

- Numeric (ou Double)
  - le type par défaut pour les nombres
  - 2, 2.3, 0, -12.8, -Inf, Inf, NaN
- Integer
  - des nombres entiers : 1, 2, 0, -1, ...
  - notation 1L, 2L, ... pour garantir qu'un nombre est un entier et non un double
  - les entiers sont un sous-ensemble des numeric
- Logical
  - deux valeurs : `TRUE` / `FALSE`
- Character (chaînes de caractères)
  - "Éthiopie", "Addis-Abeba"



Opérateurs de comparaison et logiques

Operator	Summary	Example	Result
`x < y`	`x` less than `y`	`3 < 42`	TRUE
`x > y`	`x` greater than `y`	`3 > 42`	FALSE
`x <= y`	`x` less than or equal to `y`	`3 <= 42`	TRUE
`x >= y`	`x` greater than or equal to `y`	`3 >= 42`	FALSE
`x == y`	`x` equal to `y`	`3 == 42`	FALSE
`x != y`	`x` not equal to `y`	`3 != 42`	TRUE
`!x`	not `x`	`!(3 > 42)`	TRUE
`x | y`	`x` or `y`	`(3 > 42) | TRUE`	TRUE
`x & y`	`x` and `y`	`(3 < 4) & ( 42 > 13)`	TRUE

Attention aux valeurs manquantes, `NA`



Les objets

Des valeurs et des objets peuvent être stockés dans des objets à l'aide des opérateurs d'assignation

- `<-` (Alt + -)
- `=`

```
# Stocke la valeur 4 dans x
x <- 4
x
# [1] 4
```



Les objets

Des valeurs et des objets peuvent être stockés dans des objets à l'aide des opérateurs d'assignation

- `<-` (Alt + -)
- `=`

```
# Stocke la valeur 4 dans x
x <- 4
x
# [1] 4
```

Ces valeurs peuvent être manipulées

```
x <- 4
y <- 5
z <- x + y
z
# [1] 9
```



Structures de données : vecteurs et matrices

Utilisez la fonction de combinaison `c()` pour créer un vecteur atomique

```
vecteur_numerique <- c(-12, 0, 13, NA)
vecteur_numerique
# [1] -12   0  13  NA

vecteur_caractere <- c("France", "Espagne", "Royaume-Uni", "")
vecteur_caractere
# [1] "France"      "Espagne"     "Royaume-Uni" ""
```



Structures de données : vecteurs et matrices

Utilisez la fonction de combinaison `c()` pour créer un vecteur atomique

```
vecteur_numerique <- c(-12, 0, 13, NA)
vecteur_numerique
# [1] -12   0  13  NA

vecteur_caractere <- c("France", "Espagne", "Royaume-Uni", "")
vecteur_caractere
# [1] "France"      "Espagne"     "Royaume-Uni" ""
```

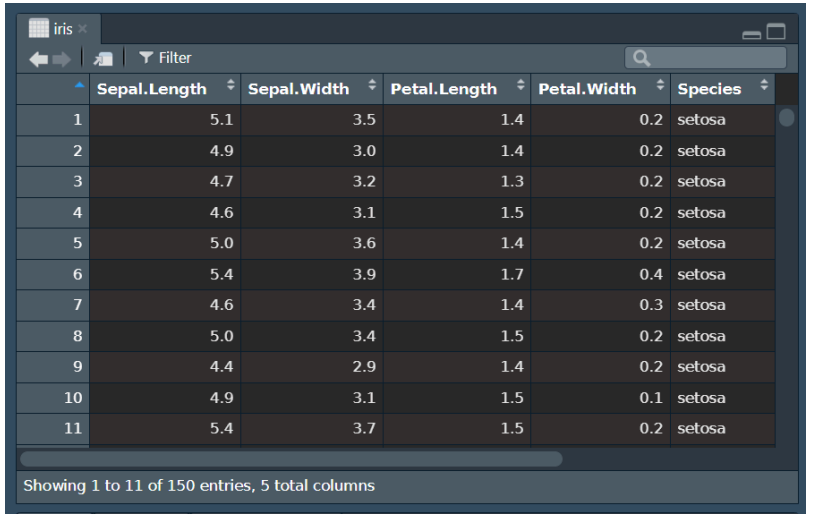
Utilisez la fonction `matrix()` pour créer une matrice

```
mat <- matrix(1:9, nrow = 3, byrow = TRUE)
mat
#      [,1] [,2] [,3]
# [1,]    1    2    3
# [2,]    4    5    6
# [3,]    7    8    9
```



Structures de données : data frames

- La plupart des jeux de données que vous utiliserez seront stockés sous forme de data frames
- Un data frame est un tableau similaire à Excel, avec des variables en colonnes et des observations en lignes
- Contrairement à une matrice, vous pouvez stocker des types de données différents dans chaque colonne



	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa
11	5.4	3.7	1.5	0.2	setosa

Structures de données : data frames

- On peut créer un data frame

```
mon_df <- data.frame(
    x = c(1, 2, 5, 7, 9, 1),
    y = c("a", "b", "c", "d", "e", "f"),
    z = c(TRUE, FALSE, TRUE, FALSE, TRUE, FALSE)
)
```



Structures de données : data frames

- On peut créer un data frame

```
mon_df <- data.frame(
    x = c(1, 2, 5, 7, 9, 1),
    y = c("a", "b", "c", "d", "e", "f"),
    z = c(TRUE, FALSE, TRUE, FALSE, TRUE, FALSE)
)
```

- Des exemples de données sont inclus dans R

```
data("iris")
iris
```



Structures de données : data frames

- On peut créer un data frame

```
mon_df <- data.frame(
    x = c(1, 2, 5, 7, 9, 1),
    y = c("a", "b", "c", "d", "e", "f"),
    z = c(TRUE, FALSE, TRUE, FALSE, TRUE, FALSE)
)
```

- Des exemples de données sont inclus dans R

```
data("iris")
iris
```

- Mais en général, vous importerez des données existantes depuis un fichier



Importer des données

Exemple avec des données de population urbaine

- Utilisez la fonction `read.csv()` pour lire un fichier CSV

```
urbanpop <- read.csv("urbanpop_1960_1966.csv", sep = ";")
```



Importer des données

Exemple avec des données de population urbaine

- Utilisez la fonction `read.csv()` pour lire un fichier CSV

```
urbanpop <- read.csv("urbanpop_1960_1966.csv", sep = ";")
```

- Elle prend différents arguments
  - un nom de fichier : "urbanpop_1960_1966.csv"
  - le séparateur entre les colonnes : ";"



Importer des données

Exemple avec des données de population urbaine

- Utilisez la fonction `read.csv()` pour lire un fichier CSV

```
urbanpop <- read.csv("urbanpop_1960_1966.csv", sep = ";")
```

- Elle prend différents arguments
  - un nom de fichier : "urbanpop_1960_1966.csv"
  - le séparateur entre les colonnes : ";"
- Le résultat de la fonction est un `data.frame`
- Si vous voulez le réutiliser plus tard, il faut le stocker dans un objet
- Ici, on le garde dans l'objet **urbanpop**



Importer des données

Exemple avec des données de population urbaine

- Utilisez la fonction `read.csv()` pour lire un fichier CSV

```
urbanpop <- read.csv("urbanpop_1960_1966.csv", sep = ";")
```

- Elle prend différents arguments
  - un nom de fichier : "urbanpop_1960_1966.csv"
  - le séparateur entre les colonnes : ";"
- Le résultat de la fonction est un `data.frame`
- Si vous voulez le réutiliser plus tard, il faut le stocker dans un objet
- Ici, on le garde dans l'objet **urbanpop**
- Vous pouvez consulter la documentation de n'importe quelle fonction avec la fonction `help()`



```
help(read.csv)
```

Les packages R

R repose beaucoup sur des packages

- Les packages R sont des collections de fonctions, de données et de code compilé
- En général, si vous devez faire quelque chose qui n'est pas possible par défaut dans R, il existe probablement un package pour ça
- Des milliers de packages sont disponibles au téléchargement sur le Comprehensive R Archive Network (CRAN) : https://cran.r-project.org
- Certains packages incluent du code venant d'autres langages (C, C++, ...) et peuvent nécessiter Rtools pour être installés



Les packages R : les utiliser

1. Installer le package (le télécharger)
   - Utilisez la fonction `install.packages()` pour télécharger la dernière version du package depuis le CRAN

```
install.packages("haven") # installe le package haven
```



Les packages R : les utiliser

1. Installer le package (le télécharger)
   - Utilisez la fonction `install.packages()` pour télécharger la dernière version du package depuis le CRAN

```
install.packages("haven") # installe le package haven
```

1. Charger le package dans R
   - Utilisez la fonction `library()` pour charger le package dans votre session R

```
library(haven)
```



Les packages R : les utiliser

1. Installer le package (le télécharger)
   - Utilisez la fonction `install.packages()` pour télécharger la dernière version du package depuis le CRAN

```
install.packages("haven") # installe le package haven
```

1. Charger le package dans R
   - Utilisez la fonction `library()` pour charger le package dans votre session R

```
library(haven)
```

- La prochaine fois que vous ouvrez R, vous n'avez pas besoin de réinstaller le package
- En revanche, vous devez le recharger avec la fonction `library()`



Les packages R : dépannage

- Le package est-il installé ? Si non, utilisez `install.packages()` pour l'installer depuis le CRAN
- Vérifiez bien que vous utilisez une chaîne de caractères entre guillemets, sinon R cherchera un objet portant le nom du package
- `install.packages()` ne fonctionne pas, mais le package existe ? Trouvez la source et suivez les instructions
- Message "Packages which are only available in source form, and may need compilation" : vérifiez que Rtools est bien installé
- Le package est-il chargé ? Si non, utilisez `library()`
- Impossible de mettre à jour un package car un autre package chargé en dépend : redémarrez RStudio



Importer des données : packages utiles

- Le package `readr` : de nombreuses fonctions pour les formats CSV, TSV, largeur fixe
- Le package `readxl` : la fonction `read_excel()` pour les formats xls et xlsx
- Le package `haven` : différentes fonctions pour SAS, SPSS, Stata
- Le package `data.table` : la fonction `fread()` pour les formats CSV, Jay, XLSX, et texte brut avec compression (.tar, .gz, .zip, .gz2, .tgz)



Les packages R : le tidyverse



Exercice 1

10:00

Nous allons utiliser les données de population par district éthiopien

- Importer les données au format Excel dans R

1. Installez le package `readxl` avec la fonction `install.packages()`
2. Chargez le package dans R avec la fonction `library()`
3. Consultez la documentation de la fonction `read_excel()` avec la fonction `help()`
4. Importez les données "ethiopia_pop.xls" dans un objet nommé **ethio_pop**



Manipulation de base des données

Data frames : caractéristiques de base

Vous pouvez facilement obtenir les informations de base sur un jeu de données avec les commandes suivantes :

```
urbanpop <- readxl::read_excel("urbanpop.xls", na = "no data")

nrow(urbanpop) # Nombre de lignes
ncol(urbanpop) # Nombre de colonnes

data_1960 <- urbanpop$year1960
data_1960

length(data_1960) # longueur d'un vecteur
```

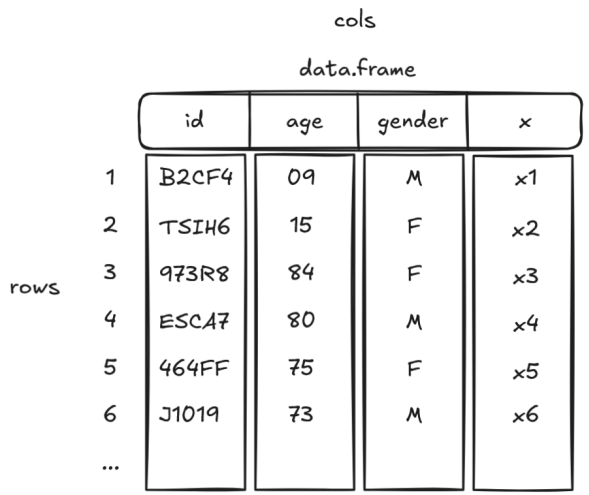


Data frames : sous-ensembles (*subsetting*)

- Vous pouvez sélectionner des éléments d'un data frame avec les crochets [ ]
- La syntaxe est

```
df[lignes, colonnes]
```

```
nouveau_df <- data.frame(
  id = c("B2CF4", "TSIH6", "973R8",
         "ESCA7", "464FF", "J1019"),
  age = c(9, 15, 84, 80, 75, 73),
  genre = c("H", "F", "F", "H", "F", "H"),
  x = paste0("x", 1:6)
)
```



Data frames : sous-ensembles (*subsetting*)

Quelques exemples

```
nouveau_df[1, 2]     # valeur de la ligne 1, colonne 2, du data frame nouveau_df
nouveau_df[1:3, 3:4] # valeurs des lignes 1 à 3, colonnes 3 à 4
nouveau_df[1, ]      # valeurs de la ligne 1, toutes les colonnes
nouveau_df[, 1]      # valeurs de toutes les lignes, colonne 1
nouveau_df[, "x"]    # valeurs de toutes les lignes, colonne nommée "x"
```



Packages utiles pour nettoyer/manipuler les données



Objectif : transformer facilement les données
Deux packages du Tidyverse



Objectif : créer des données "tidy"

Des antisèches (*cheat sheets*) sont disponibles sur GitHub

```
library(dplyr)
library(tidyr)
```

L'opérateur pipe

L'*opérateur pipe* : `%>%` ou `|>` (nouvelle version) - Cmd/Ctrl + Shift + M

1. Prend le résultat situé à gauche
2. Le transmet à la fonction suivante en tant que premier argument

```
urbanpop %>%
    select(country, year1962) %>%
    filter(year1962 > 0)
```

1. On prend nos données **urbanpop** et on les transmet à la fonction `select()` en tant que premier argument (*.data* selon l'aide)
2. On prend les nouvelles données avec les colonnes sélectionnées, et on les transmet à la fonction `filter()` en tant que premier argument (également *.data* selon l'aide)
3. On obtient les données avec les colonnes sélectionnées et les observations filtrées



L'opérateur pipe et les fonctions du tidyverse

L'opérateur pipe transmet les données au premier argument par défaut, et utilise les fonctions du tidyverse (`filter()`, `group_by()`, `summarise()` ici) pour appliquer des modifications au jeu de données initial, que l'on stocke dans un nouvel objet :

```
urbanpop_agrege <- urbanpop %>%
    filter(year1962 > 0) %>%
    group_by(region) %>%
    summarise(
        pop_moyenne = mean(year1962, na.rm = TRUE),
        pop_ecart_type = sd(year1962, na.rm = TRUE),
        pop_totale = sum(year1962, na.rm = TRUE)
    )
```



Data frames : sélectionner des variables avec dplyr

Une façon très simple de sélectionner des variables (colonnes) dans un jeu de données est la commande dplyr `select()` :

```
urbanpop1962 <- urbanpop %>%
    select(country, year1962)
# sélectionne les variables country et year1962
```



Data frames : sélectionner des variables avec dplyr

Une façon très simple de sélectionner des variables (colonnes) dans un jeu de données est la commande dplyr `select()` :

```
urbanpop1962 <- urbanpop %>%
    select(country, year1962)
# sélectionne les variables country et year1962
```

Si vous voulez sélectionner toutes les variables qui :

- commencent par le préfixe "year", vous pouvez combiner `select()` avec `starts_with()` : `select(starts_with("year"))`
- se terminent par le suffixe "_1962", vous pouvez combiner `select()` avec `ends_with()` : `select(ends_with("_1962"))`



Data frames : sélectionner des observations avec dplyr

Une façon très simple de sélectionner des observations (lignes) dans un jeu de données est la commande dplyr `filter()` :

```
urbanpop_moins100k <- urbanpop %>%
    filter(year1962 < 100000)
# tous les pays avec une population urbaine 1962 inférieure à 100k
urbanpop_moins_mediane1962 <- urbanpop %>%
    filter(year1962 < median(year1962, na.rm = TRUE))
```



Exercice 2

10:00

Dans **ethio_pop**, nous voulons comparer la population moyenne dans les woredas pastoraux et Afar, avec la moyenne dans les woredas agricoles et Amhara. Les données incluent les variables indicatrices (0 ou 1) **pastoralist**, **agricultural**, **afar** et **amhara**.

1. Sélectionnez tous les woredas pastoraux et situés en Afar (pastoralist et afar égaux à 1), et stockez les nouvelles données dans un objet nommé **past_afar**

2. Sélectionnez tous les woredas agricoles et situés en Amhara, et stockez les nouvelles données dans un objet nommé **ag_amhara**

3. À partir du data frame **past_afar**, sélectionnez la colonne **t_tl** et stockez-la dans un objet nommé **pop_pa**

4. Faites de même avec le data frame **ag_amhara**, et stockez-la dans un objet nommé **pop_aa**

5. Calculez la moyenne, la médiane, le minimum et le maximum pour les deux vecteurs

6. Comptez le nombre de woredas dans chaque vecteur

UNIVERSITÉ DE NAMUR

Bonne journée !

	jeremy.donascimento@unamur.be
.Namurgreen	Discord
	Slides